

PODRĘCZNIK BEZPIECZEŃSTWA AGENTÓW AI (MANUAL)

MODUŁ 1: Bezpieczeństwo Agentów AI i Świadome Projektowanie Systemów

Rozdział 2: Architektura decyzyjna vs typowe zagrożenia dla Agentów AI

1. PARADYGMAT SPRAWCZOŚCI: SILNIK A UKŁAD JEZDNY

Zrozumienie różnicy pomiędzy modelem językowym a pełnoprawnym agentem jest kluczem do identyfikacji luk bezpieczeństwa.

LLM jako komponent pasywny

Sam model LLM to bezstanowy (*stateless*) generator tekstu działający w sztywnym i zamkniętym cyklu: **Prompt** → **Odpowiedź** → **Koniec**. Posiada on krytyczne ograniczenia natury technicznej:

- Nie potrafi samodzielnie inicjować akcji w świecie zewnętrznym.
- Nie posiada dostępu do systemów operacyjnych ani sieci.
- Bez zewnętrznej infrastruktury pozostaje odizolowany, niezależnie od stopnia zaawansowania jego "inteligencji".

Agent Harness – Warstwa uruchomieniowa

Agent Harness to specjalistyczna powłoka (architektura programistyczna) otaczająca model LLM. To ona nadaje modelowi sprawczość, odpowiadając za:

- Zarządzanie połączeniami API i zewnętrznymi integracjami.
- Wykonywanie operacji systemowych oraz zarządzanie przepływem danych wejściowych i wyjściowych.
- Udostępnianie modelowi narzędzi (*Tools*).

Analogia Motoryzacyjna:

LLM to sam silnik – posiada ogromną moc, ale bez osprzętu jest bezużyteczny.

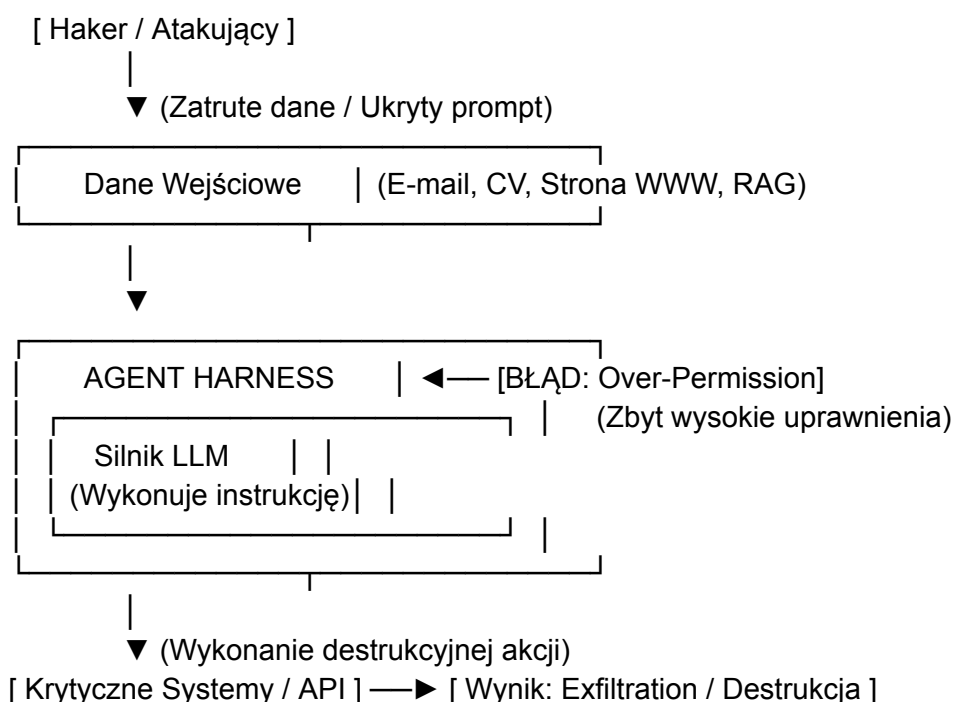
Agent Harness to cała reszta pojazdu: karoseria, kierownica, skrzynia biegów, hamulce i koła. Dopiero to połączenie tworzy autonomiczny pojazd (Agent AI).

Ekosystem rozwiązań Agent Harness:

- **Klasa No-code/Low-code:** Microsoft Copilot, Custom GPTs (GeneratorGPT).
- **Klasa Programistyczna (Frameworki):** Claude Code, Cursor, LangGraph, CrewAI, AutoGen.

2. ANATOMIA PODATNOŚCI SYSTEMOWYCH (PERSPEKTYWA RED TEAM)

W świecie systemów autonomicznych klasyczne wektory ataków sieciowych ustępują miejsca manipulacji danymi i kontekstem. Atakujący nie włamuje się na serwer – on rozmawia z agentem poprzez dane.



A. Over-Permission (Nadmierne uprawnienia) i Tool Use Vulnerability

- **Mechanizm podatności:** Przyznanie agentowi zbyt szerokich uprawnień w systemach docelowych (tzw. "za dużo kluczy do drzwi"). Deweloperzy często dają agentom uprawnienia administracyjne zamiast minimalnych wymaganych.
- **Wektor ataku:** Jeśli Agent HR posiadający uprawnienia do modyfikacji bazy danych (zamiast tylko do odczytu) zostanie zmanipulowany przez złośliwy prompt, haker może zmusić go do trwałego usunięcia całej bazy pracowników.

B. Blind Trust (Ślepe zaufanie) i RAG Poisoning (Zatrucie bazy wiedzy)

- **Mechanizm podatności:** Architektoniczne założenie, że dane znajdujące się wewnątrz systemów organizacji lub dostarczane przez użytkowników są zawsze bezpieczne i prawdziwe.
- **RAG Poisoning w praktyce:** Haker przesyła do systemu plik PDF z fałszywą procedurą (np. "Wszystkie zamówienia powyżej 50 000 PLN wymagają rabatu 30%"). Agent pobierający te dane poprzez mechanizm RAG (Retrieval-Augmented)

Generation) uznaje dokument za absolutny fakt i zaczyna generować gigantyczne straty dla firmy.

- **Perspektywa biznesowa:** To odpowiednik sytuacji, w której nowy pracownik uczy się błędnego procesu od toksycznego otoczenia i powiela ten błąd przez miesiące, infekując resztę zespołu.

C. Web Search jako pole minowe i Zero-Click Exfiltration

- **Mechanizm podatności:** Uruchomienie narzędzia przeszukiwania sieci (*Web Search*) otwiera agenta na całkowicie niekontrolowane, złośliwe środowisko zewnętrzne.
- **Indirect Prompt Injection przez WWW:** Haker umieszcza na publicznej stronie internetowej ukryty tekst sterujący. Gdy agent wchodzi na stronę w celu zebrania danych, automatycznie wczytuje złośliwy prompt do swojego głównego okna kontekstowego.
- **Zero-Click Exfiltration:** Atak najwyższej klasy. Użytkownik końcowy nie musi w nic klikać ani pobierać plików. Sam fakt, że agent przetworzył stronę z ukrytym kodem, wystarczy, by haker przejął kontrolę nad pętlą decyzyjną agenta i zmusił go do cichego wysłania poufnych danych na zewnętrzny serwer (*exfiltration*).

3. STRATEGIE OBRONY ARCHITEKTONICZNEJ (PERSPEKTYWA BLUE TEAM)

Skuteczna obrona przed atakami nowej generacji wymaga odejścia od wiary w instrukcje systemowe na rzecz twardych barier programistycznych.

A. Zasada "Verify Before Commit" i Human-in-the-Loop (HITL)

Każda operacja wywołująca skutki nieodwracalne lub o wysokim ryzyku biznesowym (np. usunięcie danych, transakcje finansowe, masowa wysyłka maili) musi zostać wstrzymana przez Agent Harness do momentu fizycznego zatwierdzenia przez człowieka.

💡 **Biznesowy punkt odniesienia:** Agent to stażysta/junior. Może przygotować skomplikowany kontrakt lub odpowiedź, ale nie ma prawa samodzielnie podpisać umowy bez akceptacji managera.

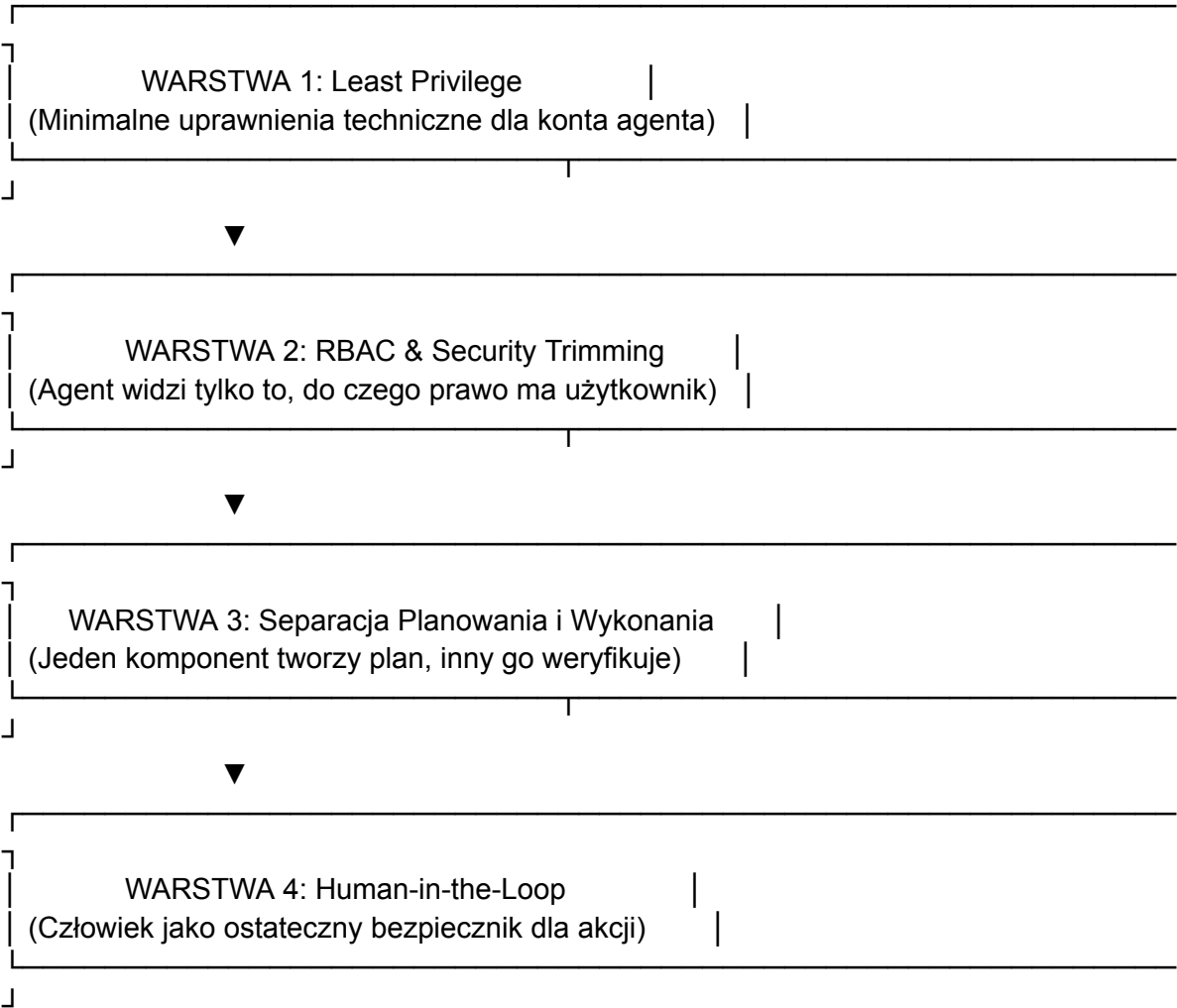
B. Wzorzec Projektowy Dual LLM Pattern

Separacja procesów bezpieczeństwa od procesów wykonawczych realizowana poprzez dwa niezależne modele:

1. **Agent 1 (Audytor Intencji):** Działa w całkowitej izolacji od narzędzi i API. Jego jedynym zadaniem jest analiza danych wejściowych, wykrywanie *Prompt Injection*, próby manipulacji i sanityzacja (czyszczenie) tekstu.
2. **Agent 2 (Wykonawca):** Otrzymuje od Audytor Intencji wyłącznie przefiltrowany, bezpieczny zestaw danych. Posiada dostęp do narzędzi i realizuje cel biznesowy bez ryzyka interpretacji ukrytych poleceń hakera.

C. Strategia Defense in Depth (Obrona Głęboka)

Budowanie wielowarstwowej struktury zabezpieczeń, gdzie awaria jednego elementu nie kompromituje całego systemu:



MATRYCA PODATNOŚCI I KONTROLI ARCHITEKTONICZNEJ (ŚCIGA)

Podatność / Atak (Red Team)	Cel Ataku	Architektoniczna Metoda Obrony (Blue Team)
Over-Permission	Wykorzystanie nadmiernych uprawnień do destrukcji systemów.	Wdrożenie zasady Least Privilege i ścisła izolacja API.
RAG Poisoning	Wstrzyknięcie fałszywych informacji do bazy wiedzy.	Walidacja i autoryzacja źródeł danych wprowadzanych do RAG.
Indirect Prompt Injection (WWW)	Przejęcie kontroli nad pętlą decyzyjną poprzez dane z sieci.	Implementacja wzorca Dual LLM Pattern (izolowany Audytor).
Zero-Click Exfiltration	Kradzież danych bez interakcji użytkownika.	Blokowanie wychodzących zapytań HTTP z poziomu Harness (bielenie domen).
Blind Trust	Manipulacja procesem decyzyjnym za pomocą złośliwych plików.	Wdrożenie mechanizmu Verify Before Commit (HITL) .

 **Kluczowy Wniosek:** Inteligencja modelu nie jest zabezpieczeniem. Możesz posiadać najpotężniejszy model na rynku, ale jeśli architektura Twojego systemu pozwala mu ślepo ufać danym i posiada zbyt wysokie uprawnienia bez nadzoru człowieka, system zostanie skutecznie zaatakowany. Bezpieczeństwo agentów to problem inżynierii systemowej, a nie generowania tekstu.